How does it work? It relies on Ruby's type coercions. Let's start at the top.

When you say `names.map(&xxx)`, you're telling Ruby to pass the `Proc` object `xxx` to the `map` method as a block. If `xxx` isn't already a `Proc` object, Ruby tries to coerce it into one by sending it a `to_proc` message.

Now `:upcase` isn't a Proc object—it's a symbol. So when Ruby sees `names.map(&:upcase)`, the first thing it does is try to convert the symbol `:upcase` into a `Proc` by calling `to_proc`. And, by an incredible coincidence, Ruby implements just such a method. If it was written in Ruby, it would look something like this:

```
def to_proc
  proc { |obj, *args| obj.send(self, *args) }
end
```

This method creates a `Proc`, which, when called on an object, sends that object the symbol itself. So, when `names.map(&:upcase)` starts to iterate over the strings in names, it'll call the block, passing in the first name and invoking its upcase method.

It's an incredibly elegant use of coercion and of closures.

Numeric Coercion

In addition to implicit and explicit object conversion, Ruby has coercion logic specific to Numeric types.

Here's the problem. When you write `1 + 2`, Ruby knows to call the + on the object `1` (an `Integer`), passing it the `Integer` `2` as a parameter. However, when you write `1 + 2.3`, the same + method now receives a `Float` parameter. How can it know what to do (particularly because checking the classes of your parameters is against the spirit of duck typing)?

The answer lies in Ruby's coercion protocol, based on the method `coerce`. The basic operation of `coerce` is simple. It takes one argument and returns a two-element array with the argument first and the original receiver of `coerce` second. The `coerce` method guarantees that the two numbers in the array will have the same class and therefore that they can be added or multiplied, compared, or whatever.

```
1.coerce(2)       # => [2, 1]
1.coerce(2.3)     # => [2.3, 1.0]
(4.5).coerce(2.3) # => [2.3, 4.5]
(4.5).coerce(2)   # => [2.0, 4.5]
```

The trick is that the coerce call happens in the reverse of the original arithmetic method—the right side of an arithmetic operation receives `coerce` with the left side of the operation as an argument to generate this array.

In other words, `1 + 2` is equivalent to the method call `1.+(2)` and Ruby calls `2.coerce(1)` to generate the array `[1, 2]` and perform the operation. This technique of calling a method on a parameter is called *double dispatch* and it allows a method to change its behavior based not only on its class but also on the class of its parameter. In this case, we're letting the parameter decide exactly *what* classes of objects should get added, compared, or whatever.

Let's say that we're writing a new class that's intended to take part in arithmetic. To participate in coercion, we need to implement a `coerce` method. This takes some other kind of